

Chapter 1

Assignment 3: RAG-Based Legal Contract Chatbot

The third stage integrates all upstream outputs — structured clauses, NER entities, SRL roles, and intent labels — into an end-to-end Retrieval-Augmented Generation (RAG) chatbot. Users can ask natural-language questions about a legal contract and receive answers grounded in specific, cited clauses.

1.1 System Overview

The system is composed of three principal modules that form a linear pipeline (Table 1.1): a vector store for dense retrieval, an LCEL RAG pipeline for answer generation, and a Streamlit front-end for user interaction.

Table 1.1: RAG system components and their roles.

Module	Technology	Role
Embedding model	<code>all-MiniLM-L6-v2</code>	Dense clause representation
Vector database	ChromaDB (cosine similarity)	Semantic clause retrieval
LLM	Gemini <code>gemini-1.5-flash</code>	Grounded answer generation
Orchestration	LangChain LCEL	Prompt + chain management
User interface	Streamlit	Interactive web application

1.1.1 Vector Store and Indexing

1.1.2 Embedding Model

Clause embeddings are produced by `sentence-transformers/all-MiniLM-L6-v2`, a lightweight 384-dimension bi-encoder optimised for semantic similarity tasks. Its small size makes it suitable for real-time encoding of query strings at inference time.

1.1.3 ChromaDB Configuration

The vector store is implemented using ChromaDB's `PersistentClient`, enabling the index to survive application restarts without re-processing the contract. Cosine similarity is used as the distance metric:

```

1 import chromadb
2 from chromadb.config import Settings
3 from sentence_transformers import SentenceTransformer
4
5 encoder = SentenceTransformer("sentence-transformers/all-MiniLM-
    L6-v2")
6
7 client = chromadb.PersistentClient(path="./chroma_db")
8 collection = client.get_or_create_collection(
9     name="legal_clauses",
10     metadata={"hnsw:space": "cosine"},
11 )

```

Listing 1.1: vector_store.py — ChromaDB collection initialisation.

1.1.4 Metadata Schema

Each document stored in ChromaDB carries the following metadata fields derived from the upstream pipeline outputs:

- `ner_entities` — JSON-serialised list of entity spans (text, label, start, end).
- `srl_predicate` — main verb of the clause.
- `srl_roles` — JSON-serialised role dictionary (Agent, Theme, Recipient, Time, Condition).
- `intent_tfidf` — intent label from the TF-IDF classifier.
- `intent_bert` — intent label from the BERT classifier.

Storing this metadata alongside the clause text enables the front-end to present NER and SRL information for each retrieved source without additional inference.

1.2 RAG Pipeline

1.2.1 LangChain LCEL Chain

The retrieval and generation steps are orchestrated via LangChain’s composable LCEL (LangChain Expression Language) syntax:

```

1 from langchain_core.prompts import ChatPromptTemplate
2 from langchain_google_genai import ChatGoogleGenerativeAI
3 from langchain_core.output_parsers import StrOutputParser
4
5 llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash",
    temperature=0.2)
6
7 chain = ChatPromptTemplate.from_messages([
8     ("system", SYSTEM_PROMPT),
9     ("human", "{question}"),
10 ]) | llm | StrOutputParser()

```

Listing 1.2: rag_pipeline.py — LCEL chain definition.

1.2.2 Anti-Hallucination System Prompt

A carefully designed system prompt constrains the model to stay within the retrieved context. The key directives are:

1. **Citation requirement.** Every factual claim must be supported by an explicit clause number (e.g. *“As stated in Clause 7, ...”*).
2. **Refusal policy.** If the answer to the user’s question cannot be found in the retrieved clauses, the model must explicitly decline to answer rather than confabulate.
3. **Scope restriction.** The model must answer only from the provided contract context and must not import external legal knowledge as a primary source.

```
1 SYSTEM_PROMPT = """You are a legal contract assistant.  
2 Answer questions ONLY based on the contract clauses provided  
   below.  
3  
4 Rules:  
5 - Cite clause numbers for every claim (e.g. "Clause 3 states  
   that...").  
6 - If the answer is NOT found in the provided clauses, respond:  
7   "I cannot find information about this in the contract."  
8 - Do NOT use external legal knowledge as a primary answer source  
   .  
9  
10 Contract clauses:  
11 {context}  
12 """
```

Listing 1.3: System prompt enforcing citation and refusal.

1.2.3 Multi-Turn Conversation

The `chat()` method maintains a list of `HumanMessage` and `AIMessage` objects to support multi-turn dialogue. The conversation history is prepended to each new request, allowing the model to resolve coreferences across turns (e.g. *“Who does they refer to in the previous answer?”*).

1.2.4 Top- k Retrieval

The number of retrieved clauses is configurable via the `top_k` parameter (default: 5). At query time, the user’s question is embedded with the same `all-MiniLM-L6-v2` encoder, and the k clauses with the highest cosine similarity scores are returned from ChromaDB and injected into the system prompt as the context window.

1.3 Streamlit User Interface

1.3.1 Interface Layout

The web application (`app.py`) is built with Streamlit and organised into two panels:

- **Sidebar.**
 - Top- k slider (range 1–10) to control retrieval breadth.
 - *Build Index* button: triggers contract parsing and ChromaDB indexing.
 - *Clear Index* button: deletes the persisted collection to allow re-indexing with a different document.
- **Main panel.**
 - Chat input box for natural-language questions.
 - Conversation history rendered as alternating user/assistant message bubbles.
 - Colour-coded intent badges next to each retrieved source clause (Obligation = blue, Prohibition = red, Right = green, Termination Condition = orange).
 - Expandable *Sources* panel for each assistant response, revealing the retrieved clause text along with its NER entities and SRL roles extracted in the upstream pipeline.

1.3.2 Index Building Workflow

When the user clicks *Build Index*, the application executes the following steps:

1. Load the contract text from the configured file path.
2. Run the full upstream pipeline (clause splitting, NER, SRL, intent classification).
3. Embed all clause texts with `all-MiniLM-L6-v2`.
4. Upsert embeddings, texts, and metadata into the ChromaDB collection.
5. Display a success notification with the total number of indexed clauses.

Once the index is built, query latency is dominated by embedding the question (≈ 10 ms) and the Gemini API round-trip, making the chatbot responsive for interactive use.